

Introduction

I. Project outcome

- The project focused on home security automation to detect theft, intruders, and fires. Key outcomes included:
- Designing a network security system for both interior and exterior monitoring
- Collecting and transferring data in real-time with minimal errors
- Detecting intruders, unauthorized entry, and fire hazards using computer vision and machine learning, while minimizing false alarms
- Developing a user-friendly website to log and display data

I. Project pipeline

The project focuses on automating a full-fledged home security system, covering both the interior and exterior of the house.

Exterior Security:

CCTV cameras are installed at key points around the house. Computer vision techniques are used for automation:

- Object Detection: Moving objects are detected using Non-Max Suppression, and HOG (Histogram of Oriented Gradients) descriptors are used for human detection.
- Face Recognition: Captured faces are checked against known family member images using OpenCV's Haar Cascade and DeepFace for face verification. If a recognized family member is detected, no alerts are triggered. If an intruder is detected, motion is tracked using `cv2.createBackgroundSubtractorMOG2`, which detects changes by subtracting the background.

Interior Security:

Non-invasive sensors (temperature, light, sound, CO2, and PIR) are used to detect intruders. These sensors are adjusted based on the house size:

- Intruder Detection: A machine learning model, based on a PyTorch deep learning neural network, processes the sensor data. If the system detects a sudden increase in occupancy or abnormal sensor activity, alerts are sent to the homeowner.
- Anomaly Detection: Isolation Forest is used to detect abnormal sensor readings, such as changes indicating a house fire.

System Integration:

Both the exterior and interior systems are integrated to form a comprehensive home security system. The combined solution provides automated alerts and real-time monitoring for any suspicious activity.

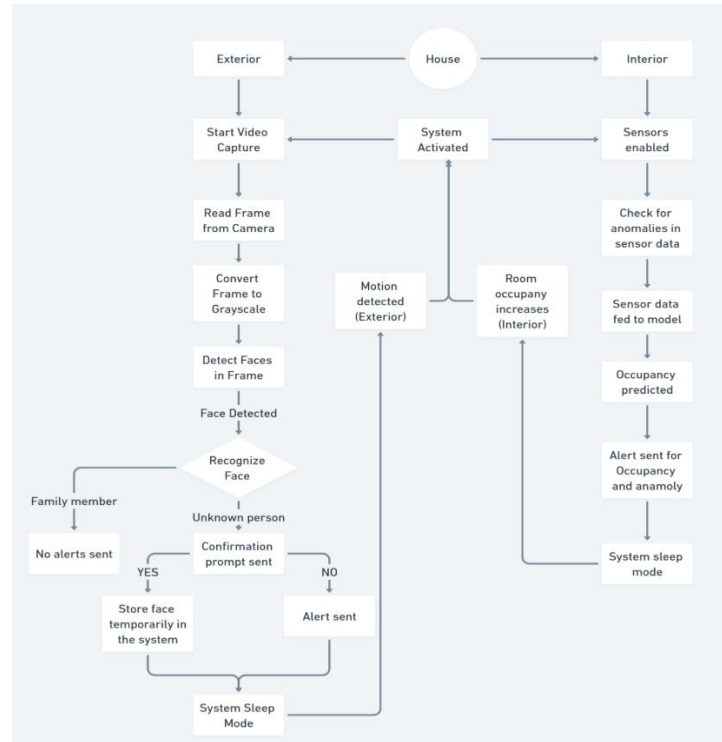


Fig 1. Logic of the automated security system

II. Hardware connections

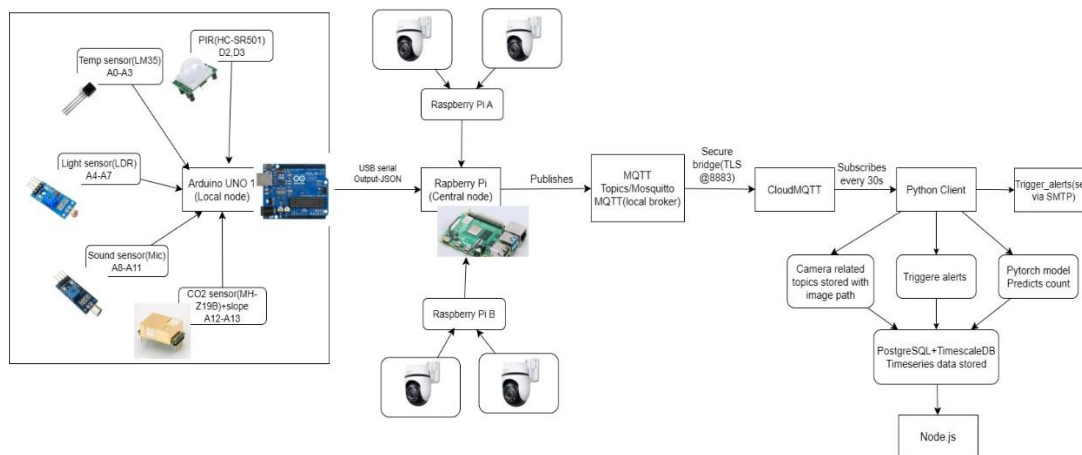


Fig.2 Backend of system

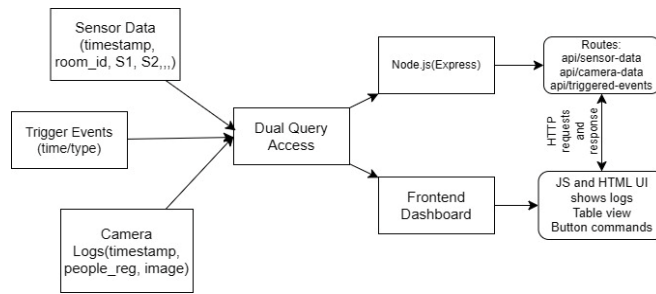


Fig.3 Frontend of system

A) Sensor → local Arduino:

Each room with a sensor suite was connected to a common Arduino node to collect data, a typical room of size 6m x 4.6m utilizes a combination of Sensors which includes S1_Temp, S2_Temp, S3_Temp S4_Temp, S1_Light, S2_Light, S3_Light, S4_Light, S1_Sound, S2_Sound, S3_Sound, S4_Sound, S5_CO2 S5_CO2_Slope, S6_PIR, S7_PIR.

Table 1 : Type of sensor and its connection with the arduino

Sensor	Type	Quantity	Pins
Temperature	Analog (LM35)	4	A0-A3
Light	Analog (LDR + resistor), Voltage divider	4	A4-A7
Sound	Analog (mic sensor), Analog output to ADC	4	A8-A11
CO ₂ + Slope	Analog (MH-Z19B)	1	A12, A13
PIR	Digital (HC-SR501)	2	D2, D3

With the change of room size, a certain number of sensors can be reduced or increased.

B) Raspberry Pi Communication(Central node):

The Raspberry is considered the central node, combining the exterior and interior segment.

The Arduino is coded to print sensor data in JSON format, Raspberry listens to the USB serial port), it reads JSON, parses it, and prepares it for further processing. The option for choosing GPIO-to-GPIO UART was not chosen because of the voltage mismatches, raspberry uses 3.3V logic and Arduino UNO uses 5V logic a voltage divider or level shifter must be used, so the alternate was chosen which was USB

On the other hand, the CCTV footages are captured. If any motion is detected in the cameras with (cv2.createBackgroundSubtractorMOG2), the footages are immediately sent for review to the central node.

Step 1: Capture CCTV feeds, using IP cameras

Step 2: Motion detection (MOG2) (Raspberry Pi A and B)

Step 3: Object/Human Detection (HOG + NMS): OpenCV's cv2.HOGDescriptor() + SVM people detector, apply Non-Maximum Suppression to refine detection (Raspberry Pi A and B)

Step 4: Face Detection (cv2.CascadeClassifier) (Raspberry Pi A and B)

Step 5: Face Recognition (DeepFace.verify) (Central Raspberry node)

Step 6: The prediction pushed to cloud.

C) Transferring messages with MQTT:

The final sensor data and the Raspberry Pi camera module data are stored in the central node. The MQTT runs with the logic of subscription and publishing of messages to topics

Table 2: Topics in MQTT

Data Type	Topics	Format
Sensor Data (Room 1)	home/room1/sensors	JSON
Sensor Data (Room 2)	home/room2/sensors	JSON
Motion Detected	home/security/motion/raspA/camera1	Boolean/String
Human Detected	Home/security/human/raspA/camera1	Boolean/String
Face Verified	home/security/face/verified	JSON (with ID, status)
Face Unknown	home/security/face/unverified	JSON + cropped image path/url

The data from the central node is pushed to the server via the MQTT local broker Mosquitto to bridge it to CloudMQTT, so any data that is published locally can be published in the cloud. Encryption is added via the command “bridge_cafile /etc/ssl/certs/ca-certificates.crt” and is run in the port 8883.

D) Using and storing data from CloudMQTT → ML model and PostgreSQL

Python client in the model subscribes to CloudMQTT topics, incoming JSON data every 30 sec is parsed and is passed to the pretrained PyTorch-based deep learning neural network model as a test file with the same number input features utilized for training for prediction. The prediction (room_occupancy_count) is combined with the sensor data and stored in PostgreSQL. The timescaleDB creates a time series data in PostgreSQL, that is the sensor data with the time series and prediction is stored in tables. Continuous aggregate view feature is to summarize the data over time in chunks, and a retention policy is set up to store only one month's data.

E) Creating a simple dashboard to retrieve the data

A simple dashboard using Node.js was built with JavaScript and HTML. To retrieve the data requested by the user.



Fig 4. Web dashboard

III. Tools Used

Hardware:

- 1) 3x Raspberry Pi
- 2) 1x Arduino uno
- 3) 4x LM35
- 4) 2x HC-SR501
- 5) 1x MH-Z19B
- 6) 4x mic sensor
- 7) 4x LDR + resistor

Software:

- 1) Arduino IDE
- 2) ThingSpeak
- 3) MQTT explorer
- 4) Timescaledb PostgreSQL
- 5) PyCharm/VSCode
- 6) Pytorch
- 7) Javascript
- 8) HTML and CSS

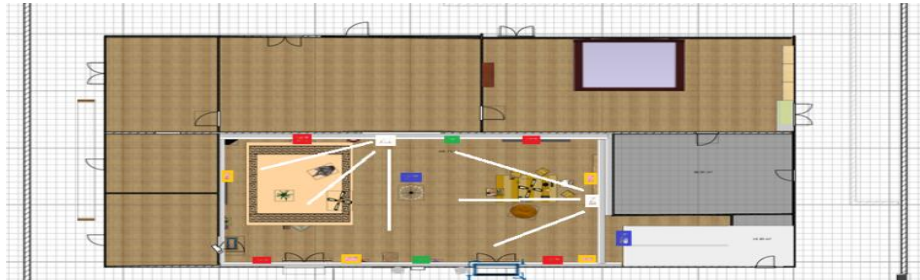
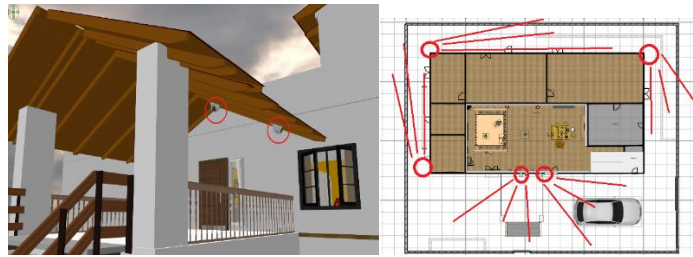


Fig.5 CCTV and Sensor placement